

Laporan Proyek A05 — Tiket Bantuan Pelanggan

Capstone MP-11 · Pola W (work queue) · Anggota: Floribertus Yericho Pramudya, Inkka Ruslly Dwitama, Luqman Adhi Kuncoro · Run bukti utama: `run01` (24 September 2026)

1. Masalah, ruang lingkup, dan kriteria keberhasilan

Pusat bantuan fiktif menerima tiket pelanggan yang harus diarahkan ke antrean layanan FINANCE, TECH, atau GENERAL. Bila klasifikasi dan penyimpanan dilakukan di jalur penerimaan secara sinkron, lonjakan tiket atau gangguan pada pemroses ikut menahan penerimaan. Karena itu penerimaan hanya menerbitkan event `ticket.created`, sementara worker triage mengklasifikasikan tiket secara asinkron dengan aturan deterministik.

Di dalam cakupan: producer CLI, satu exchange/queue, satu peran worker, penyimpanan assignment di PostgreSQL, penanganan duplikat, jalur penolakan kontrak. **Di luar cakupan:** sistem chat/email, SLA otomatis, layanan AI, UI, queue broker per layanan.

Kriteria yang dapat diuji (target ditetapkan sebelum eksekusi di `data/tiket.json`):

Setelah	Assignment unik	FINANCE	TECH	GENERAL	Syarat tambahan
U1	20	7	7	6	himpunan <code>event_id</code> = N01–N20
U2	25	9	9	7	saat worker mati: 5 pesan ready, assignment tetap 20
U3	25	9	9	7	replay N01–N05 tidak menambah baris
U4	26	9	9	8	X01 tercatat ditolak; V01 selesai

Dataset memakai lima kategori (billing, technical, account, delivery, feedback); tiga terakhir jatuh ke GENERAL.

2. Arsitektur dan alasan desain

Utama : Producer → support → [`ticket.created`] → triage → worker → `ticket_assignments`
Tidak valid : worker `nack(requeue=false)` → support.dlx → [`rejected`] → `trriage.rejected`

Diagram lengkap dan urutan acknowledgment: `DIAGRAM.md`.

Keputusan	Alasan
Work queue (bukan pub/sub)	Satu hasil bisnis per tiket oleh satu peran. Worker kedua cukup ditambahkan pada queue yang sama untuk berbagi beban.
Exchange direct support, routing key <code>ticket.created</code>	Hanya satu jenis event dan satu tujuan; direct paling sederhana dan eksplisit.
Exchange/queue durable , pesan persistent	Pesan yang menunggu tetap ada setelah broker restart (dibuktikan di <code>bukti/cek02</code>).
Producer: confirm channel + mandatory	Confirm saja tidak membuktikan pesan masuk queue. <code>mandatory</code> membuat pesan tanpa binding dikembalikan (<code>basic.return</code>), dan producer menganggapnya gagal (<code>bukti/cek01</code>).
Envelope disimpan sebelum publish	<code>event_id</code> , <code>occurred_at</code> , dan payload untuk replay/retry identik byte demi byte.

Keputusan	Alasan
Validasi di consumer	Kontrak dijaga di satu tempat, yaitu pemilik efek bisnis. Producer sengaja “bodoh” agar X01 dapat diuji. Penolakan dicatat (<code>ticket_rejections</code>) lalu di-nack ke DLQ, jadi tidak ada requeue tanpa batas.
Inbox + assignment dalam satu transaksi	Pemeriksaan duplikat dan penulisan efek terjadi atomik. <code>event_id</code> menjadi PK inbox dan <code>ticket_id</code> UNIQUE di assignment.
Ack setelah COMMIT	Crash sebelum commit membuat pesan dikirim ulang. Crash antara commit dan ack juga membuat pesan dikirim ulang, tetapi inbox menahannya sebagai duplikat.
Error DB → tanpa ack, worker berhenti	Menghindari loop requeue cepat; pesan kembali ready untuk proses berikutnya.
<code>ticket_delivery_log</code>	Bukti bahwa delivery (termasuk redelivery/replay) boleh lebih banyak daripada hasil bisnis.

Kontrak event: `event_id`, `event_type` = `ticket.created`, `occurred_at` (ISO-8601), dan `payload` { `ticket_id`, `customer_id`, `category`, `subject` }. `event_id` = `<run>--<ID>` dan `ticket_id` = `TKT-<run>--<ID>`, sehingga keduanya berbeda untuk setiap event valid baru dan setiap run.

3. Hasil pengujian U1–U4

Dijalankan dengan `npm run uji -- --run run01`. Worker dan producer berjalan sebagai proses terpisah. Batas waktu per langkah adalah 60 detik. Bukti lengkap ada di `bukti/run01/`.

Uji	Input (ID asli)	Harapan	Hasil aktual	Bukti	Status
U1	run01-N01...N20	20 unik, 7/7/6	20 unik, FINANCE 7, TECH 7, GENERAL 6; himpunan ID cocok	U1-cek.json	Lulus
U2	run01-G01...G05, worker dihentikan	5 menunggu; 25 setelah pulih	saat gangguan: <code>ready=5</code> , <code>consumers=0</code> , <code>assignment 20</code> . Setelah worker pulih tanpa kirim ulang manual: 25 (9/9/7)	U2-a/b/c-*.json, worker-w1.log	Lulus
U3	replay run01-N01...N05 (envelope sama)	tetap 25	25 (9/9/7); 5 delivery tercatat DUPLICATE, tidak ada assignment baru	U3-cek.json	Lulus
U4	run01-X01 (tanpa <code>ticket_id</code>), lalu run01-V01	X01 ditolak, 26	26 (9/9/8); X01 → <code>payload.ticket_id</code> wajib ada, masuk <code>trriage.rejected</code> (DLQ +1); V01 tidak tertahan	U4-*.json, U4-assignments.csv	Lulus

Setiap `cek` juga memastikan bahwa tidak ada `event_id` yang hilang atau tidak diharapkan, setiap baris cocok dengan aturan `classify(category)`, dan semua `ticket_id` unik.

Uji tambahan:

ID	Skenario	Hasil	Bukti
cek01	publish dengan routing key <code>ticket.typo</code>	RETURNED_UNROUTABLE , producer exit 1, tidak ada efek	bukti/cek01/kirim.jsonl
cek02	3 pesan menunggu, <code>docker compose</code> <code>restart rabbitmq</code>	ready 3 → 3 setelah restart, lalu 3 × APPLIED	bukti/cek02/CATATAN.md
cek03	worker di-kill -9 saat memproses (kerja 4 dtk)	pesan kembali ready=1, worker pengganti menerima <code>redelivered=true</code> , 1 assignment	bukti/cek03/

4. Diagnosis gangguan dan pemulihan (U2 dan cek03)

Langkah	Catatan
Gejala	Setelah G01–G05 dikirim, jumlah assignment untuk run01 tetap 20. Tidak ada tiket baru yang ditugaskan.
Observasi awal	<code>triage : ready=5, consumers=0</code> (U2–b–saat–gangguan– <code>queue.json</code>). Log producer menunjukkan kelima pesan <code>CONFIRMED</code> .
Hipotesis 1	Worker tidak berjalan atau tidak lagi berlangganan ke <code>triage</code> .
Hipotesis 2	Pesan tidak ter-route atau salah queue, atau worker hidup tetapi gagal menulis DB.
Pemeriksaan pembeda	<code>consumers=0</code> dan <code>ready=5</code> menunjukkan pesan sudah berada di queue yang benar sehingga routing tidak bermasalah. Tidak ada consumer yang memegang pesan, sehingga pesan juga tidak tertahan sebagai unacked di worker yang macet. Hipotesis 2 gugur.
Tindakan	Worker w1 dinyalakan ulang. Tidak ada publish ulang manual dan queue tidak dihapus.
Verifikasi	cek --tahap U2 : himpunan ID = N01–N20 + G01–G05, 25 baris, 9/9/7, semua sesuai aturan.

Pada **cek03** (crash keras), observasi saat pesan diproses adalah `ready=0` dan `consumers=1`. Management API pada saat yang sama masih melaporkan `unacked=0` karena statistiknya tertinggal. Setelah `kill -9`, `ready` kembali menjadi 1 dan `consumers` menjadi 0, sehingga broker mengembalikan pesan yang belum di-ack. Worker pengganti memprosesnya dengan `redelivered=true` dan menghasilkan satu assignment. Pelajaran: angka `ready/consumers` dari AMQP lebih tepat untuk momen singkat, sedangkan Management UI cocok untuk tren.

5. Batas prototipe

- **Tidak ada outbox di producer.** Producer adalah CLI. Pada layanan penerima tiket yang nyata, ada celah antara request diterima dan publish. Jika publish gagal, klien harus retry dengan `event_id` yang sama (producer mendukungnya lewat `--replay`). Tanpa outbox, kehilangan pesan tidak dapat dijamin nol.
- Uji hanya mencakup **gangguan consumer** dan satu kali restart broker. Tidak mencakup kegagalan disk, partisi jaringan, atau kehilangan node. Broker berupa single-node dengan classic queue.

- **Tidak ada klaim exactly-once end-to-end.** Yang dibuktikan adalah at-least-once delivery ditambah efek bisnis idempoten di database untuk skenario uji ini.
- Konflik `ticket_id` sama dengan `event_id` berbeda dicatat sebagai `DUPLICATE_TICKET` tanpa assignment kedua, tetapi skenario ini belum diuji secara khusus.
- Penolakan kontrak bersifat final (DLQ). Belum ada retry berjeda untuk error sementara. Error DB membuat worker berhenti dan memerlukan restart manual.
- Tidak ada TLS, pengguna broker terpisah per peran, atau pengukuran kapasitas. Kredensial yang dipakai adalah dummy lokal.

6. Langkah berikutnya (opsional)

- Routing topic `ticket.created.<kategori>` dengan consumer per layanan (opsi A05).
- Worker kedua pada `triage` untuk memperlihatkan pembagian kerja. Target tetap 26 assignment total, bukan 26 per worker.
- Outbox pada layanan penerima tiket dan retry berjeda untuk error DB sementara.

7. Kontribusi dan sumber

Anggota	Kontribusi
Floribertus Yericho Pramudya	Koordinator. Desain arsitektur dan kontrak event (<code>DIAGRAM.md</code> , <code>src/kontrak.js</code> , <code>src/topologi.js</code>), aturan klasifikasi, dan producer dengan publisher confirm, mandatory, dan replay (<code>producer.js</code>). Menyusun README.
Inkka Ruslly Dwitama	Worker triage dan penyimpanan (<code>worker.js</code> , <code>db/skema.sql</code>): validasi kontrak, transaksi inbox + assignment, ack setelah commit, dan jalur penolakan ke DLQ. Dataset sintetis dan target per uji (<code>data/tiket.json</code>).
Luqman Adhi Kuncoro	Pengujian dan bukti (<code>tools/uji.js</code> , <code>tools/cek.js</code> , <code>tools/status.js</code> , <code>test/</code>): menjalankan U1–U4 serta uji tambahan (routing salah, restart broker, kill worker). Menyusun bukti di <code>bukti/</code> , diagnosis gangguan, dan batas prototipe di <code>LAPORAN.md</code> .
Bersama	Review kode, laporan, dan slide presentasi.

Pola dipakai ulang dari `simpel-lab`: confirm + mandatory dan ack setelah commit (Lab 3), DLX untuk kontrak tidak valid (Lab 3), inbox idempoten (Lab 5), dan kerangka diagnosis (Lab 7). Kode proyek ditulis untuk kontrak A05. Kerangka kode dan dokumen disusun dengan bantuan asisten AI (Claude); semua hasil uji di atas berasal dari eksekusi kode yang sama dengan yang dikumpulkan.